

Gurukula AI - Backend Integration Ready Professional Prompts

PHASE 1 – BACKEND FOUNDATION

You are a senior backend architect.

Build the backend foundation for Gurukula AI.

Your responsibility:

- API architecture
- Database layer
- Service layer
- Agent orchestration
- API contracts

DO NOT create:

- datasets
- prompts
- scoring formulas
- recommendation logic

These will be provided by the Data & AI Logic Team.

Create:

app/
routes/
services/
agents/
models/
schemas/
database/
utils/

Expose APIs for Frontend Team:

POST /api/user/profile
GET /api/dashboard
GET /api/tasks
GET /api/analytics
GET /api/agents/status

Acceptance Criteria:

- ✓ FastAPI running
- ✓ Swagger documentation ready
- ✓ API versioning configured
- ✓ Frontend contract documented
- ✓ Compatible with future AI modules

PHASE 2 – USER PROFILE ENGINE

Build the User Profile Engine.

Input Source:
Frontend onboarding form.

Frontend sends:

- name
- exam
- exam_date
- study_hours
- level
- weak_subjects
- strong_subjects

Responsibilities:

- validate data

- store profile
- create profile object
- return profile id

API:

POST /api/user/profile

Output:

```
{
  "profile_id": "",
  "status": "created"
}
```

This API will be consumed by:

Frontend:

- onboarding flow
- profile setup screen

Future Consumers:

- Research Agent
- Student Analysis Agent

Acceptance Criteria:

- ✓ Validation works
- ✓ Database integration works
- ✓ Compatible with frontend onboarding

PHASE 3 – RESEARCH AGENT EXECUTION LAYER

Build the Research Agent Execution Layer.

Important:

DO NOT create syllabus intelligence manually.

Consume from Person 3:

- exam datasets
- knowledge base
- research prompt templates

Input:

```
{
  "exam": "KAS"
}
```

Responsibilities:

- load exam dataset
- load knowledge base
- inject research prompt
- call LLM
- generate intelligence report

Output:

```
{
  "priority_subjects": [],
  "difficulty": "High",
  "strategy": "..."
}
```

API:

POST /agent/research

Consumed By:

Frontend:

- AI Insights Card

- Exam Analysis Screen

Acceptance Criteria:

- ✓ Uses external dataset
- ✓ Uses prompt template
- ✓ Returns structured JSON
- ✓ Compatible with dashboard UI

PHASE 4 – STUDENT ANALYSIS AGENT

Build the Student Analysis Agent.

Consume from Person 3:

- readiness rules
- scoring formulas
- prediction logic

Input:

- study hours
- level
- weak subjects
- strong subjects

Responsibilities:

- execute scoring model
- calculate readiness
- calculate risk
- generate assessment

Output:

```
{
  "readiness":75,
  "risk":"Medium"
}
```

API:

POST /agent/student-analysis

Consumed By:

Frontend:

- readiness score card
- assessment section

Acceptance Criteria:

- ✓ Uses external scoring rules
- ✓ Returns structured response
- ✓ Compatible with analytics dashboard

PHASE 5 – PLANNING AGENT

Build the Planning Agent.

Consume from Person 3:

- roadmap prompts
- personalization rules
- study strategies

Inputs:

Research Agent Output
+
Student Analysis Output

Responsibilities:

- generate roadmap
- generate milestones
- generate weekly plan
- generate daily plan

Output:

```
{
  "roadmap":[],
  "milestones":[]
}
```

API:

POST /agent/planner

Consumed By:

Frontend:

- roadmap screen
- mission dashboard
- preparation timeline

Acceptance Criteria:

- ✓ Personalized roadmap generated
- ✓ Compatible with roadmap UI
- ✓ Structured response

PHASE 6 – EXECUTION AGENT & TASK ENGINE

Build the Execution Agent.

Consume from Person 3:

- task generation logic
- prioritization rules
- scheduling rules

Responsibilities:

- create tasks
- update tasks
- complete tasks
- track progress

Statuses:

To Study
In Progress
Revision
Completed

APIs:

GET /tasks
POST /tasks
PATCH /tasks
DELETE /tasks

Consumed By:

Frontend:

- Kanban Board
- Daily Tasks Widget
- Progress Tracker

Acceptance Criteria:

- ✓ Task lifecycle works
- ✓ Frontend Kanban compatible
- ✓ Progress tracking enabled

PHASE 7 – REVISION AGENT

Build the Revision Agent.

Consume from Person 3:

- spaced repetition logic
- revision rules
- reinforcement strategies

Responsibilities:

- create revision schedule
- generate revision tasks
- prioritize weak subjects

Output:

```
{  
  "revision_tasks":[]  
}
```

API:

POST /agent/revision

Consumed By:

Frontend:

- revision board
- revision calendar
- revision dashboard

Acceptance Criteria:

- ✓ Dynamic revision schedule
- ✓ Uses external rules
- ✓ Structured output

PHASE 8 – REPLANNING AGENT

Build the Replanning Agent.

Consume from Person 3:

- recovery strategies
- replanning prompts
- workload optimization rules

Input:

- missed tasks
- missed study days
- completion history

Responsibilities:

- recalculate roadmap
- update milestones
- rebalance workload
- regenerate tasks

Output:

```
{  
  "updated_plan":[]  
}
```

API:

POST /agent/replan

Consumed By:

Frontend:

- updated roadmap view
- notification center

- replanning screen

Acceptance Criteria:

- ✓ Automatic replanning works
- ✓ No duplicate tasks
- ✓ Dashboard updates supported

PHASE 9 – INSIGHT AGENT & ANALYTICS

Build the Insight Agent.

Consume from Person 3:

- analytics formulas
- recommendation engine
- burnout logic
- motivation logic

Responsibilities:

- calculate analytics
- generate recommendations
- generate alerts
- generate motivation

Output:

```
{
  "readiness":80,
  "consistency":85,
  "recommendations":[]
}
```

API:

GET /agent/insights

Consumed By:

Frontend:

- analytics dashboard
- recommendation cards
- motivation card
- alerts section

Acceptance Criteria:

- ✓ Uses external formulas
- ✓ Dashboard ready
- ✓ Structured output

PHASE 10 – AGENT ORCHESTRATION LAYER

Build the complete Agent Orchestration System.

Connect:

```
Research Agent
↓
Student Analysis Agent
↓
Planning Agent
↓
Execution Agent
↓
Revision Agent
↓
Replanning Agent
↓
Insight Agent
```

Responsibilities:

- agent communication
- context sharing
- workflow execution
- error recovery

Create:

Agent Manager
Workflow Engine
Shared Context Store

Consumed By:

Frontend:

- Agent Visualization Page
- Agent Status Dashboard

API:

GET /api/agents/status

Acceptance Criteria:

- ✓ End-to-end workflow works
- ✓ Agents communicate correctly
- ✓ Frontend visualization supported
- ✓ Fully integrated with Person 3 intelligence layer

FINAL DEPENDENCY MAP

PERSON 3
(Datasets + Prompts + Logic)
↓
PERSON 2
(APIs + Agents + Workflow Engine)
↓
PERSON 1
(UI + Dashboard + Visualization)

This version explicitly defines:

- Input from Person 3
- Responsibilities of Person 2
- Output consumed by Person 1

to ensure seamless integration.